Architecture Review Framework

A reusable documentation and review method, applied to the group’s MateMate design.

The framework connects three review questions. What are the system boundaries? Who can change its important state? What evidence supports the current design? It uses C4 for structural views, arc42 for explanations and three required records for checks that can be repeated after a change.

Required views and sections

Source	Part adopted	Deliverable and MateMate location
C4 L1	System context	Scope diagram and the human interaction (p. 3).
C4 L2	Container	One runnable desktop application and its boundary (p. 3).
C4 L3	Component	Seven group subsystems and their allowed dependencies (p. 4).
arc42 §1	Introduction and goals	Purpose, scope, stakeholders and quality priorities (pp. 3, 5).
arc42 §2	Constraints	Offline execution, local interaction and chess rules (p. 3).
arc42 §3	Context and scope	External actor, interface and excluded features (p. 3).
arc42 §§4-5	Strategy and building blocks	Threading, responsibilities and interfaces (pp. 3-4).
arc42 §§6-7	Runtime and deployment	Three runtime scenarios and the installation topology (pp. 3, 5).
arc42 §9	Architecture decisions	Decisions, trade-offs and revisit triggers (p. 4).
arc42 §§10-11	Quality, risks and debt	Acceptance scenarios, scorecard and open risks (pp. 5-7).

Where the framework improves the review

C4 provides structural diagrams. This framework adds decision rationale, state ownership and testable quality gates alongside those views. It builds on arc42’s documentation sections by prescribing a fixed sequence of views, a consistent diagram key and records for state authority, change impact and evidence expiry.

The added records take time to maintain. A pilot could measure how quickly reviewers locate a writer, forecast the impact of a change and find current evidence, and check their answers against the same design.

How to produce and review it

Give views, decisions and checks stable identifiers so a reviewer can follow them across the document.

Review procedure

Step	Action	Output
1. Set scope	Agree on stakeholders, system boundary, constraints and measurable quality scenarios.	Context, goals and Q IDs
2. Decompose	Draw the context, containers and relevant components. Record allowed dependencies and describe each interface.	Views, component IDs and edge list
3. Walk through	Trace a normal flow, a failure and a proposed change. Record decisions, state authority and expected impact.	Runtime traces, ADRs, A and P IDs
4. Verify	Bind checks to a named baseline, run them at the required maturity and evaluate missing evidence.	Evidence register and gate result
5. Maintain	Repeat affected checks and probes after changes. Update views and the evidence baseline before sign-off.	New baseline and updated results

Three required additions

Record	Minimum fields and review rule
Authority ledger (A)	State item, sole writer, readers, commit boundary, invariant and rejection behavior. Require one declared writer per critical state item.
Change-impact probe (P)	Change, required and permitted blocks, protected contracts, touch budget and observed diff. Check the changed set and the protected contracts.
Evidence and expiry register (E)	Claim, evidence class, check, baseline, result and invalidation trigger. Expire evidence when a recorded input changes. An open required check blocks release.

Diagram notation

Channel	Meaning
Fill color and role	Green A = application logic. Blue T = technical input. Amber 0 = generic graphics. Gray = person or aggregate. Text repeats the role.
Frame style	Double = owns committed state. Single = internal element without that authority. Dashed = external participant. The outer dotted line encloses the system.
Box size and priority	P1 large = correctness, turn or latency critical. P2 standard = supporting. P3 small = optional. Assign priority within the view.
Arrow	A labeled arrow shows an allowed caller-to-provider dependency in a component view or an interaction in a context or container view.

MateMate · Context and deployment

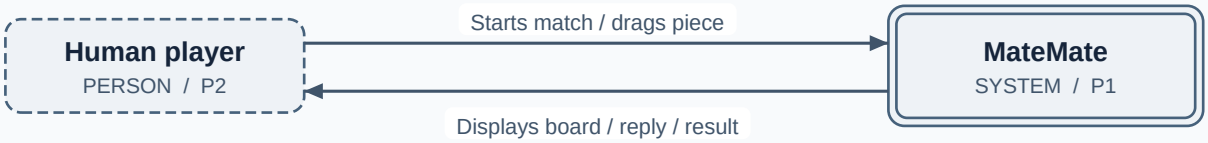
Scope, stakeholders and C4 levels 1 and 2.

Goals and constraints

The player needs legal moves and a responsive computer reply on a drag-and-drop chessboard. Maintainers need to be able to change the evaluator and desktop toolkit through known interfaces. Course reviewers need to trace the group's make-move scenario and check the service ownership and allowed dependencies.

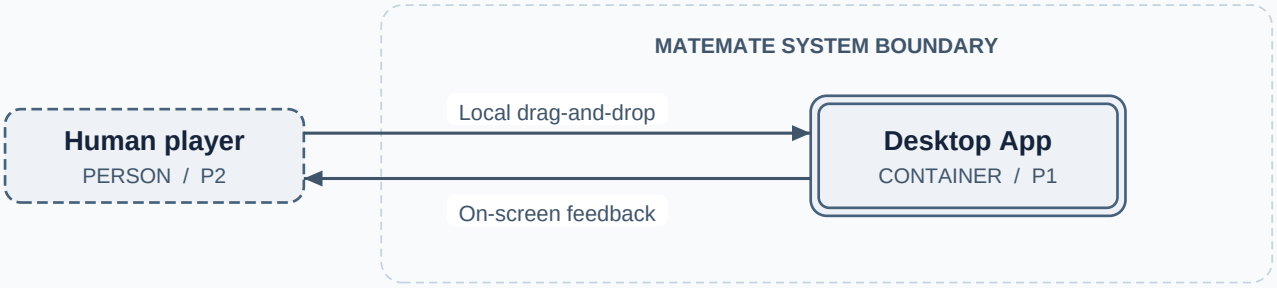
The design assumes one active offline match. The board and history live in memory. Online play, saved games, tournament clocks and a threefold/fifty-move claim interface are outside this iteration. Automatic fivefold and seventy-five-move draws use the match history, with checkmate taking precedence.

C4 level 1 · System context



The human is the only external actor. MateMate accepts match and move actions and presents board state, invalid-move feedback and game results.

C4 level 2 · Containers

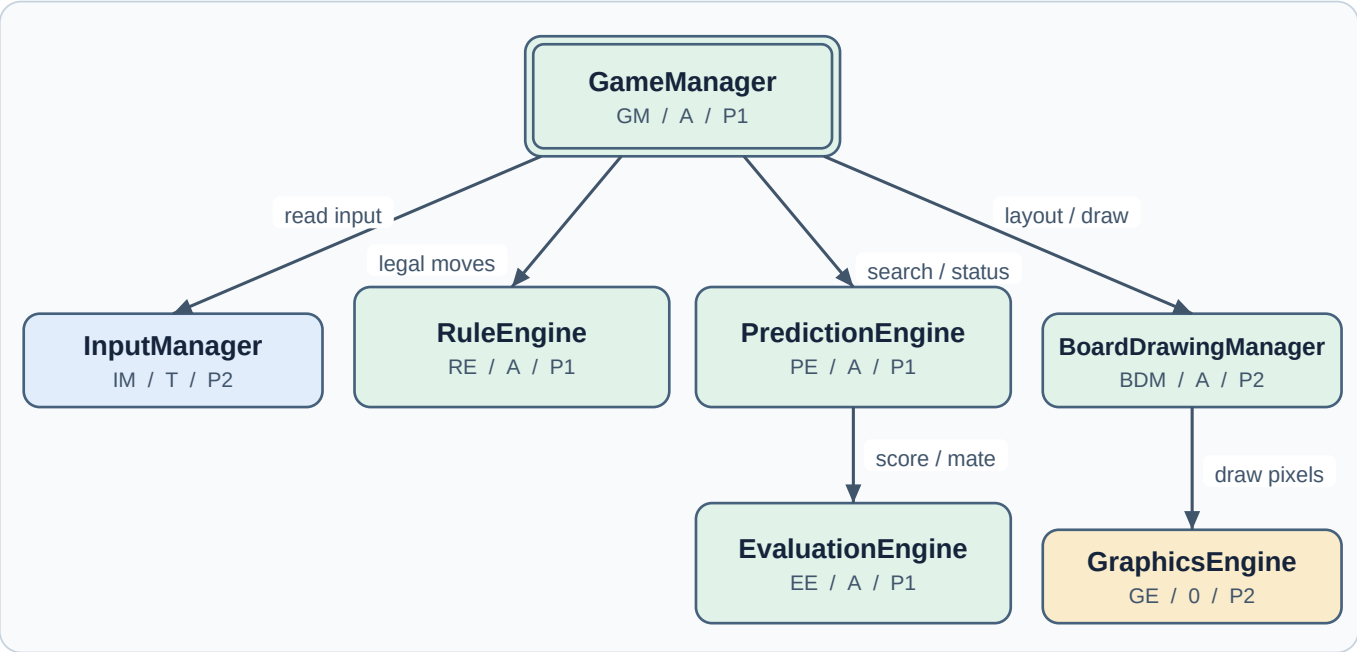


The proposed deployment is one Java desktop process with a JavaFX input and graphics boundary. GameManager owns the in-memory match and runs a search worker within the same process. The worker and GUI exchange immutable messages. The application and compatible runtime are installed on the player's workstation.

Key. Gray marks the actor or aggregate. Dashed marks the external player. Double marks the state-owning aggregate. P1 marks a review-critical system or container and P2 a supporting element. Arrows show the labeled interactions.

MateMate · Components and decisions

C4 level 3 inside the Desktop App. Names and allowed dependencies follow the group's classwork.



Key. Green A = chess logic. Blue T = technical input. Amber 0 = generic pixel graphics (the group's 0 classification). Double border = committed-state owner. Large P1 = review-critical. Standard P2 = supporting. Arrows point from caller to allowed provider.

Contracts. GM reads raw events from IM and obtains layout and rendering from BDM. GM calls RE for legal transitions and PE for search and position status. BDM draws via GE. PE's sole dependency is EE for scores and mate evaluation. GM resolves PE's requests for legal successors or rule facts through RE and returns immutable data.

Architecture decisions

Decision	Choice and trade-off	Revisit when
ADR-01 **Position and history**	Keep immutable positions and accepted history inside GM. Copying simplifies search branches and rejection. In-place apply and undo would reduce allocation.	Profiling shows that snapshot allocation causes the Q3 latency target to be missed.
ADR-02 **Rule facts for search**	GM supplies legal successors and rule facts to a resumable PE search. PE evaluates through EE. Mediating expansion adds coordination and preserves the group's dependency table.	Profiling shows that the request/response loop causes a latency problem.
ADR-03 **Commit model**	GM writes live state on a serialized command loop. A worker returns candidates tagged with match ID, revision and token.	A second local or remote writer becomes a requirement.

The proposal's service table maps all 20 elicited services to these subsystems. Shared inputs, board positions and rule facts are values passed through the pictured interfaces.

MateMate · Runtime and quality

The group’s “make a move” criterion anchors the acceptance checks.

Runtime scenarios

Scenario	Cooperation and data
R1 · Human move and reply	GM reads the raw IM events and uses the BDM layout to map (824, 535) to e4. With e2 as the press square, GM asks RE to apply e2-e4, commits the position and renders via BDM and GE. GM drives PE’s search, supplies RE transitions and facts on request, and PE uses EE to score and detect mate. GM validates the reply with RE, commits it and renders it through BDM and GE.
R2 · Castle	GM supplies a snapshot to RE. RE checks the matching right, home pieces, clear path and safe king squares. A valid castle changes both pieces in one transition. GM commits one move. A rejection preserves the board, history and revision.
R3 · Timeout or stale result	GM runs PE on a worker with a proposed 750 ms search budget. At the deadline GM uses the best completed result or a prevalidated legal fallback. A restart or committed reply retires the active token. Results with a stale match ID, revision or token cannot commit.

Quality scenarios and acceptance

ID	Stimulus and measurement	Pass condition
Q1 Correctness	Test one e2-e4 exchange with a fixed legal reply, 12 castling fixtures across both colors (legal, lost rights, blocked and attacked paths), mate for each color and one stalemate.	All 16 outcomes match. Check board and history for valid moves and verify rejected moves add no entries.
Q2 Integrity	Inject a stale result after restart, a duplicate result and a result for an incorrect revision. Compare the move history and revision.	All three injected results cause zero commits.
Q3 Responsiveness	Warm the release build and run 100 fixed position-and-move pairs leaving a nonterminal engine turn at standard difficulty on a recorded 4-core/16 GB workstation. Measure accepted drop to first paint of the legal reply.	Nearest-rank p95 at most 1,000 ms. Every run yields exactly one legal reply.
Q4 Changeability	Run probes P1 and P2 separately. Compare affected production blocks and test the protected contracts.	Both probes respect their budget, touch the required blocks and preserve the protected contracts.

These are proposed implementation checks. The fixture counts, search budget and latency target extend the group’s make-move criterion.

MateMate · Review records

D2 is the design baseline aligned with the group's classwork.

A · Authority ledger

Critical state	Sole writer and readers	Commit rule
A1 · Live board, rights and revision	GM writes. RE and PE receive position values. BDM receives render snapshots.	Commit a legal transition at the expected match ID and revision.
A2 · Accepted moves and repetition counts	GM writes and seeds the worker with a copy.	Update history and counts with A1. Rejected moves add nothing.
A3 · Active phase and search token	GM writes. The worker receives a token value.	Retire the token at restart or accepted reply. Reject stale results.
A4 · Game-level input history	GM writes. IM supplies raw event samples.	Store the raw drop and map the click using the matching BDM layout. A cancelled drag cannot mutate A1 or A2.

P · Change-impact probes

Probe	Required and permitted blocks	Protected contracts and forecast
P1 · Revise the positional heuristic	{EE}. Touch budget of one block.	Keep PE-facing score and mate contracts. Forecast {EE}.
P2 · Replace the desktop toolkit	{IM, GE}. Touch budget of two blocks. Record platform packaging changes separately.	Keep raw input and pixel drawing interfaces, GM square mapping and BDM layout. Forecast {IM, GE}.

E · Evidence and expiry register

ID / claim	Check and baseline	Status and expiry trigger
E0 · Design alignment	Compare D2 with the group table: seven subsystems, six allowed dependencies, 20 service assignments and A1-A4.	Reviewed at design level. Expires if the service, dependency or authority map changes.
E1 / Q1 · Correctness	R1 and R2 fixtures plus EE mate and stalemate cases on a named build.	Pending. Repeat after move flow, rules, evaluation, history or fixture changes.
E2 / Q2 · Integrity	Inject stale and duplicate results against a named build and fixture revision.	Pending. Repeat after scheduling, commit or fixture changes.
E3 / Q3 · Latency	Run the fixed workload on a named build, settings and workstation.	Pending. Repeat after a relevant build, workload, settings or machine change.
E4 / Q4 · Change	Compare probe diffs and contract checks pinned to before and after revisions.	Pending. Repeat after protected interface or probe definition changes.

Record the build, test or probe revision and relevant configuration for each result. Exclude expired evidence until the check runs again.

MateMate · Architecture scorecard

Structural checks refer to the design. Implementation checks require results from a named build.

Metric	Calculation and release target	MateMate at D2
Dependency conformance	Allowed observed edges ÷ all observed direct edges. Target 1.00. Inspect the design now and compare imports and calls before release.	6/6 = 1.00 on the design graph.
Cycle freedom	1 - (blocks in directed cycles ÷ all blocks). Count each block once, including self-loops. Target 1.00.	1 - 0/7 = 1.00 on the design graph.
Service assignment coverage	Group services assigned to their specified owner ÷ all 20 group services. Target 1.00. Check each service number against the group's list.	20/20 = 1.00 in the proposal's service table.
State authority coverage	Critical state items with exactly one declared writer ÷ all inventoried critical state items. Target 1.00. Verify actual writes in code.	4/4 = 1.00 in the authority ledger.
Change isolation	Passing probes ÷ executed probes. A pass touches the required blocks, stays within the budget and preserves protected contracts. Target 1.00 with both probes run.	Pending. Forecast one block for P1 and two for P2.
Current acceptance evidence	Required check groups with current passing implementation evidence ÷ all required check groups. Target 1.00.	0/4 evidence coverage. Q1-Q4 are pending.
Timely legal replies	Cases with exactly one legal reply painted within 1,000 ms ÷ 100 reference cases. Target ≥ 0.95 and all replies legal.	Pending. Record p95 latency in ms alongside the ratio.

A structural metric with an empty denominator is not applicable. Release requires current passing evidence for Q1-Q4.

Risks and planned checks

Search cost. Snapshot copies and mediated expansion may reduce the depth reached within the time budget. Profile E3 and track fallback frequency. **Rule-state drift.** Use RE for live and hypothetical transitions and include repeated-position fixtures when implementing automatic draws.

Sources

MateMate group classwork, *Services*, 3 November 2026 (SWA_Services_261103.docx).

Martin Hasitschka, *Architecture Documentation*, V704, PDF pp. 5, 9-10 and 72.

Martin Hasitschka, *Architecture Development*, V702, PDF pp. 27-36.

C4 model diagrams and notation.

arc42 documentation, sections 1-12.

FIDE Laws of Chess, arts. 3.8-3.9 and 9.2-9.6.